

Informe de Ingeniería y Arquitectura de Software — Sistema de Automatización IDIC

Proyecto: Automatización de Informes de Integridad Científica

Institución: Instituto de Investigación Científica (IDIC), Universidad de Lima

Autor: Arquitecto de Software Senior / Ingeniero Principal

Versión: 3.0 (Documentación Exhaustiva)

Fecha: 28 de abril de 2026

1. Resumen Ejecutivo

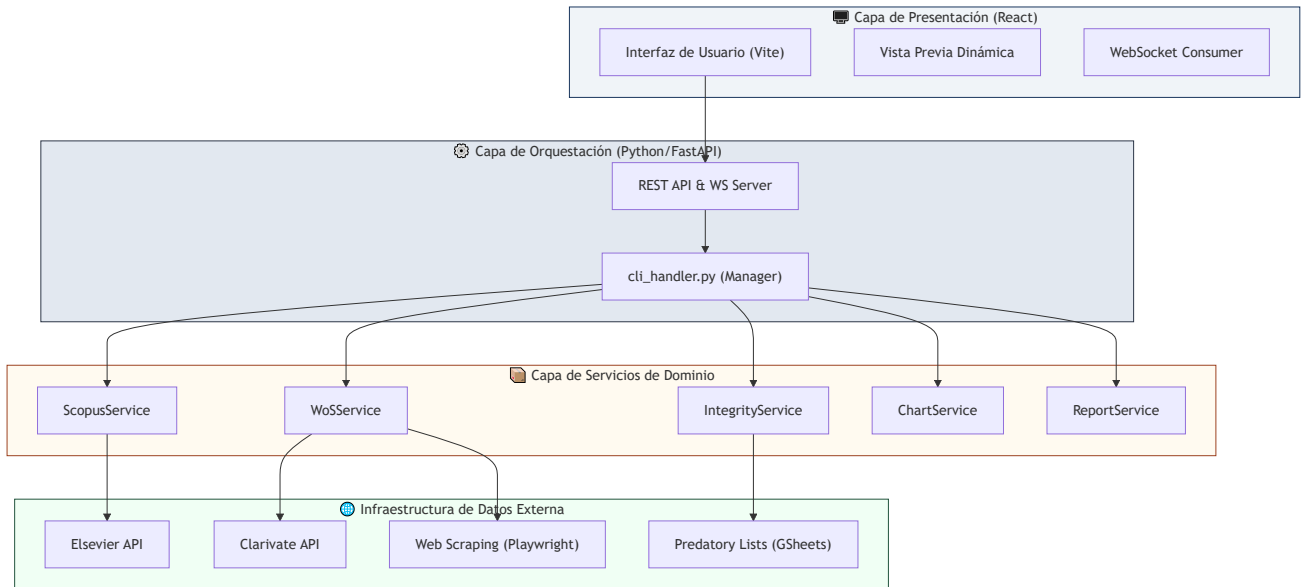
Este informe técnico documenta de manera exhaustiva la arquitectura, los procesos de ingeniería y la lógica de negocio del **Sistema de Automatización de Informes de Integridad Científica**. El sistema ha sido diseñado para mitigar la carga operativa del equipo de integridad del IDIC mediante la orquestación de servicios asíncronos y la generación automatizada de artefactos legales y técnicos.

La solución v3.0 se destaca por su **diseño modular basado en micro-servicios internos**, lo que permite una mantenibilidad superior y una capacidad de respuesta en tiempo real mediante WebSockets.

2. Arquitectura de Referencia

2.1 Visión Global del Sistema

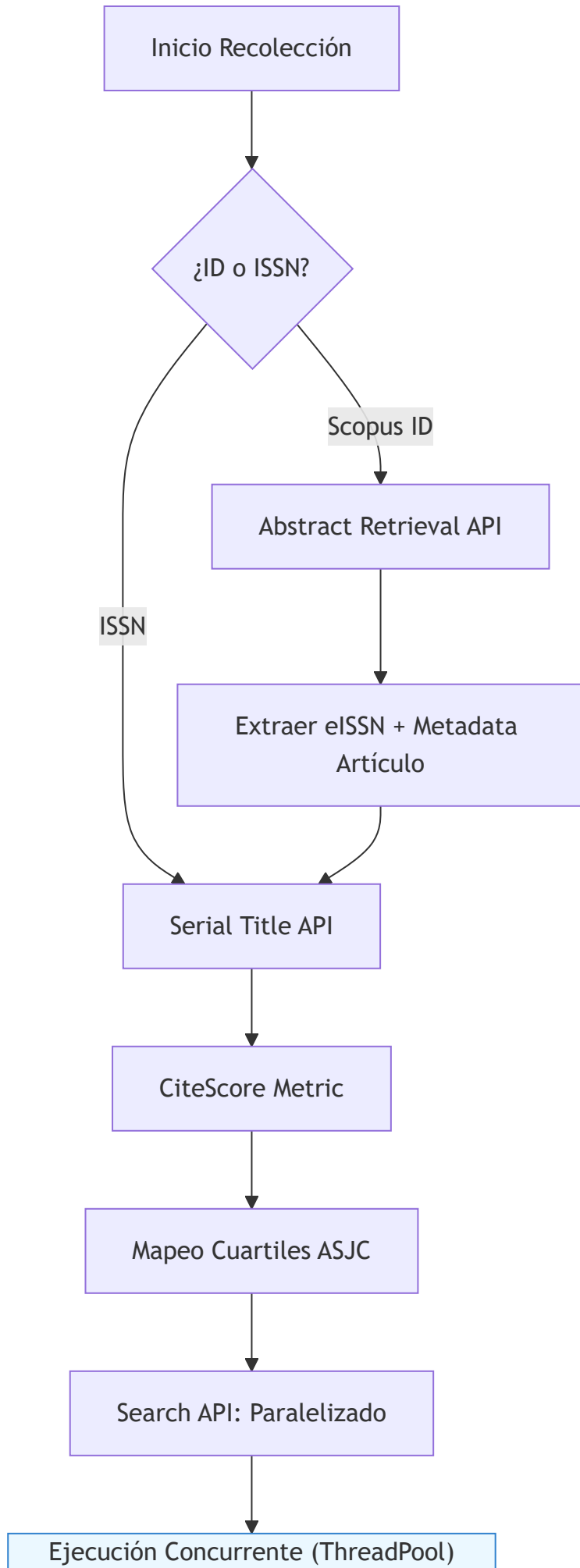
El sistema opera bajo un modelo de **Orquestación Centralizada**. El backend de FastAPI actúa como el punto de entrada, delegando la ejecución a un controlador CLI que maneja el ciclo de vida de los servicios de dominio.

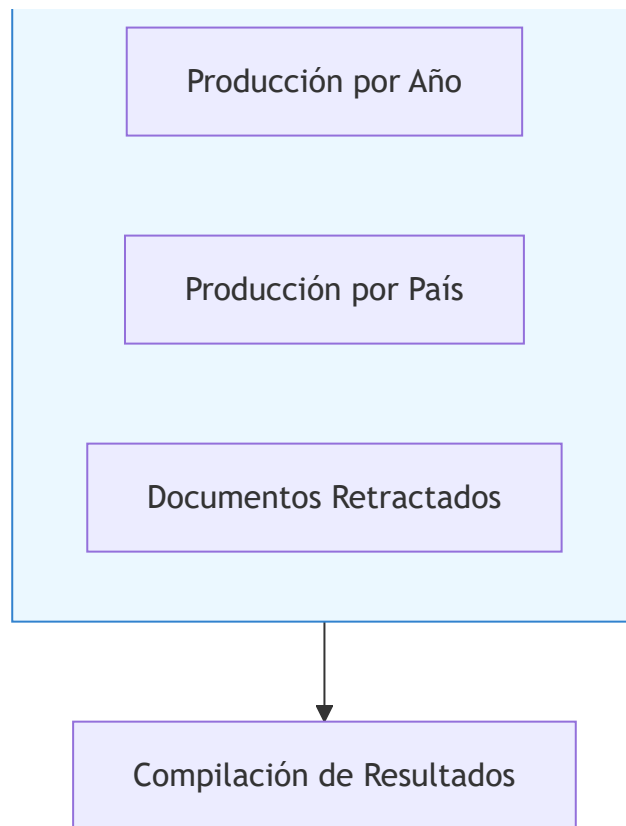


3. Desglose Detallado de Procesos

3.1 Proceso de Recolección Bibliométrica (ScopusService)

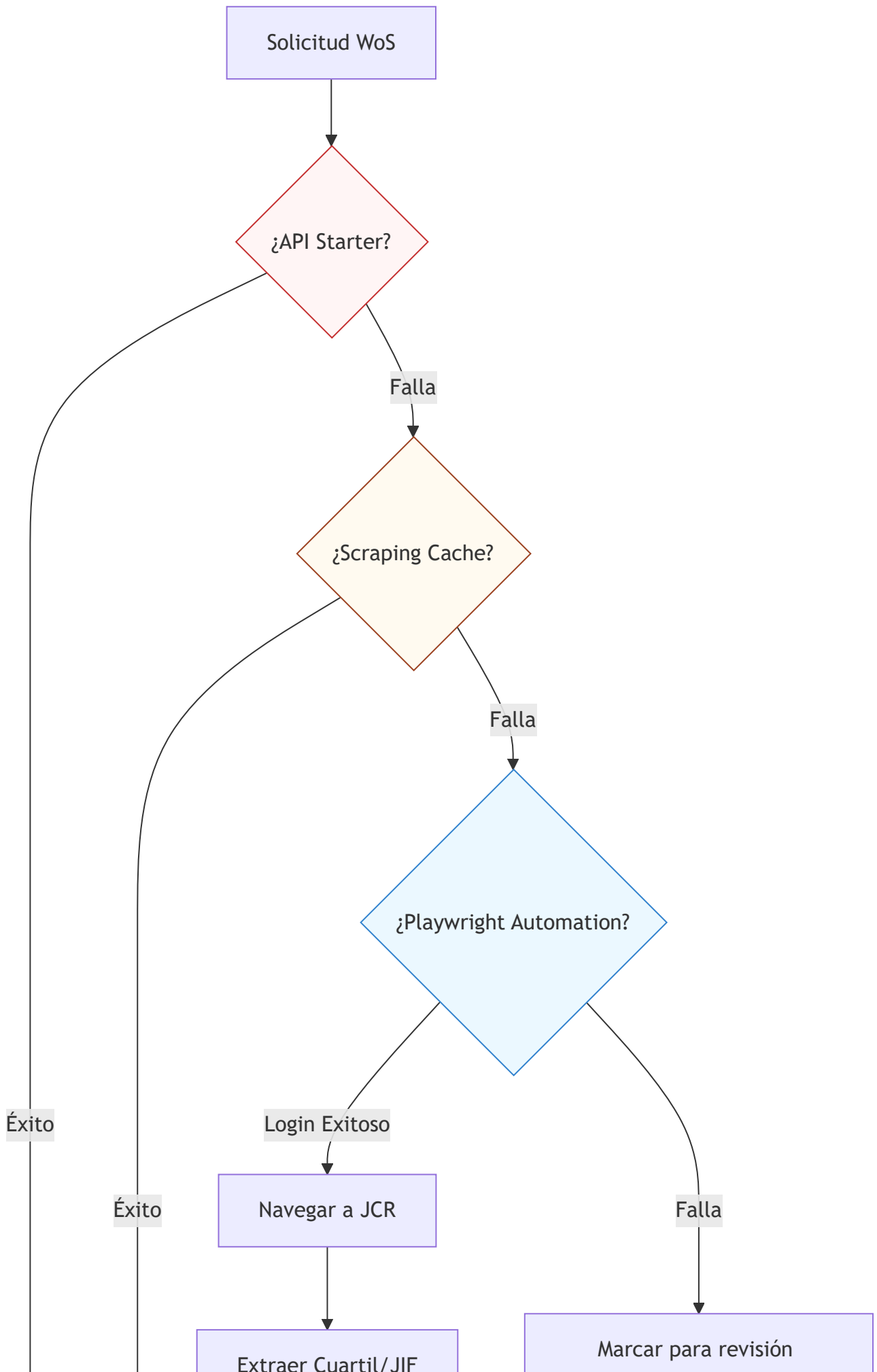
La extracción en Scopus utiliza un flujo de tres etapas para maximizar la obtención de datos con el menor número de llamadas a la API posible.

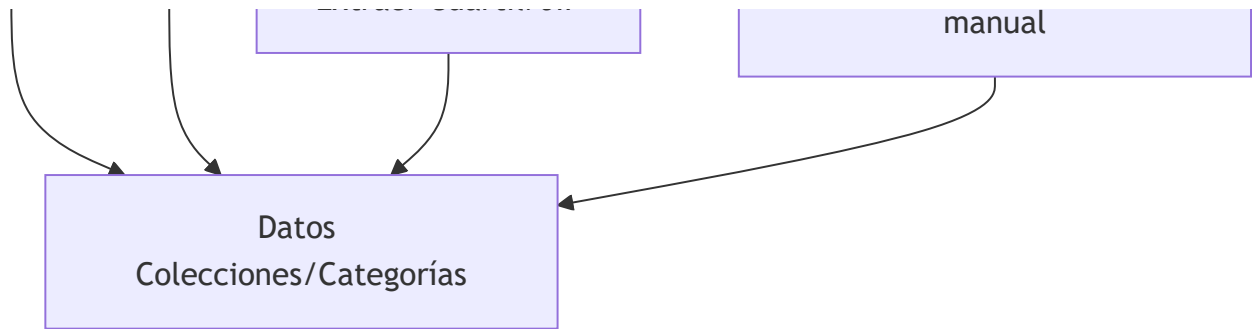




3.2 Estrategia de Fallback WoS y JCR (WoSService)

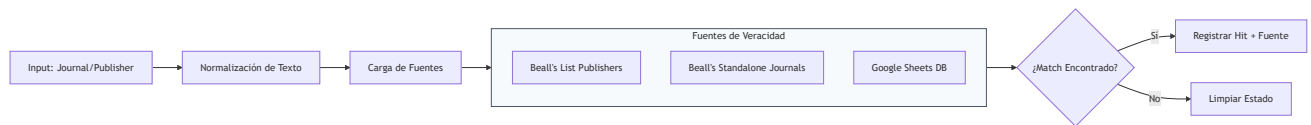
Uno de los mayores desafíos técnicos es la obtención de datos de Web of Science. Hemos implementado un sistema de **Resiliencia Adaptativa**.





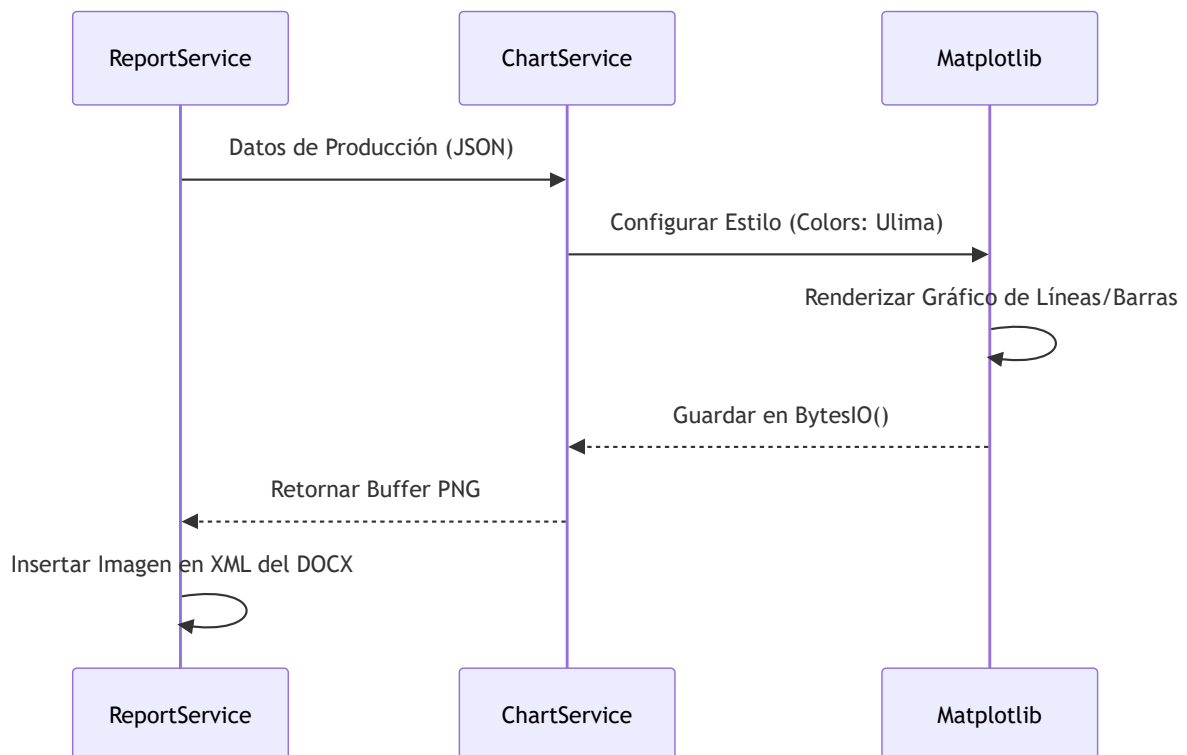
3.3 Validación de Integridad (IntegrityService)

Este servicio aplica heurísticas de comparación de strings para detectar coincidencias en listas de editoriales y revistas sospechosas.



3.4 Generación de Artefactos Visuales (ChartService)

Para garantizar la integración perfecta en el documento Word, los gráficos se generan como flujos binarios (ByteStreams) en memoria.



3.5 Ensamblado de Documentación (ReportService)

El ensamblado final utiliza el motor `python-docx` para manipular el XML del estándar OpenXML.

Etapa	Acción Técnica
Cabecera	Inserción de estilos de párrafo "Times New Roman" bold.
Tablas	Formateo de bordes y sombreado condicional para cuartiles.
Gráficos	Ajuste de dimensiones (9.0 x 4.0 in) para evitar desbordamiento.
Placeholders	Marcado de campos [COMPLETAR] con color de resaltado amarillo.

4. Stack Tecnológico Detallado

Componente	Tecnología	Rol Estratégico
Engine	Python 3.11	Lenguaje base para procesamiento de datos y APIs.
API	FastAPI	Framework ASGI de alto rendimiento para WebSockets.
Automation	Playwright	Navegación headless para sitios con renderizado JS complejo.
Frontend	React 18	Gestión de estado para el preview interactivo.
Styling	Vanilla CSS	Control total sobre la tipografía institucional (Inter/Times).

5. Gestión de Errores y Trazabilidad

El sistema implementa un protocolo de comunicación tipado sobre WebSockets:

- STEP** : Notifica el avance de la barra de progreso.
- DATA** : Envía fragmentos de información para el preview parcial.
- INFO/WARN** : Mensajes operativos para el usuario.
- ERROR** : Captura excepciones de red o de parseo, permitiendo la recuperación del sistema sin reinicio.

6. Conclusiones Arquitectónicas

La arquitectura v3.0 del Sistema de Automatización IDIC representa una solución de **Ingeniería de Software Senior** que prioriza la robustez sobre la simplicidad. Al segmentar la lógica en servicios especializados y proporcionar una capa de orquestación resiliente, el sistema no solo cumple con su objetivo actual, sino que está preparado para integrar futuras fuentes de datos con un impacto mínimo en el código base existente.

